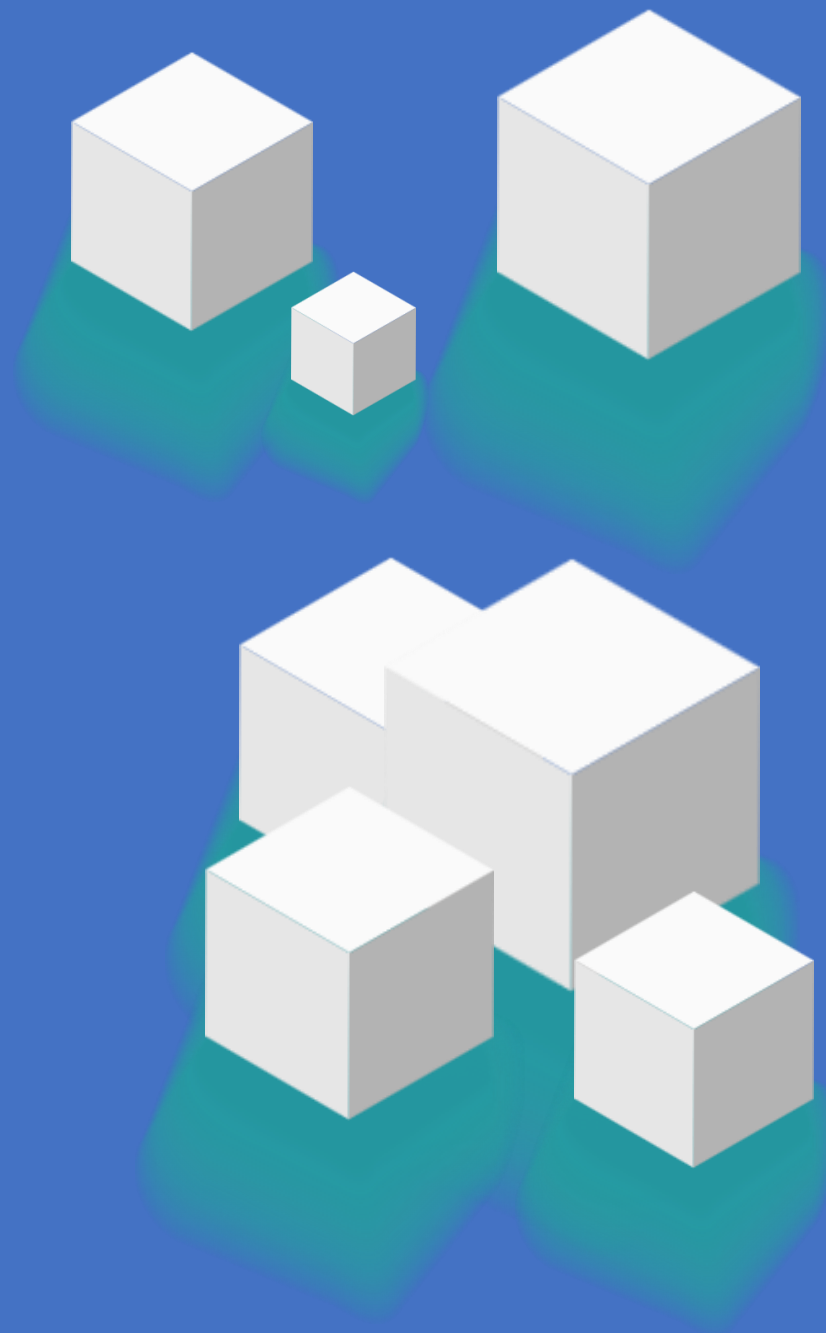
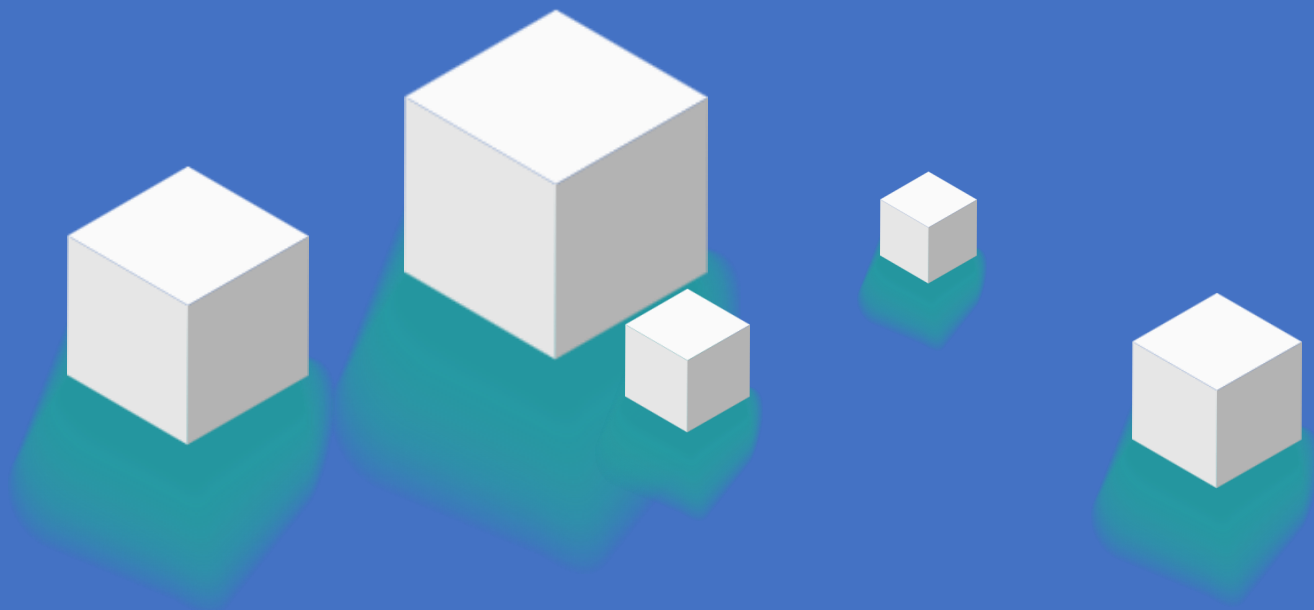


强化学习与高频探索



>> 今天的学习目标

强化学习与高频探索

- 什么是强化学习

CASE: 搭建RL交易环境

CASE: cartpole AI

- Q-Learning原理

CASE: 迷宫问题

- DQN原理

CASE: DQN择时策略

- 智能拆单

大单怎么买

TWAP 与 VWAP

PPO的核心思想

市场冲击模型

CASE: 智能拆单环境

CASE: TWAP与RL拆单对比

什么是强化学习

为什么需要强化学习

传统策略的痛点：

Ethan 是一个量化交易员。他之前用技术指标（MA 金叉买入、死叉卖出）做交易，遇到了几个问题：

- MACD 说买入，RSI 说超买该卖出 —— 指标之间经常矛盾
- 参数（比如均线周期 5/10/20）需要人工调优 —— 不同市场状态需要不同参数
- 回测效果好，实盘不行 —— 参数过拟合历史数据

=> Ethan 想：能不能让一个 AI Agent 自己从数据中学习什么时候该买、什么时候该卖？（不用人去设定规则）

=> 这是强化学习 (Reinforcement Learning) 要解决的问题。

为什么需要强化学习

Thinking: 交易决策的本质是什么?

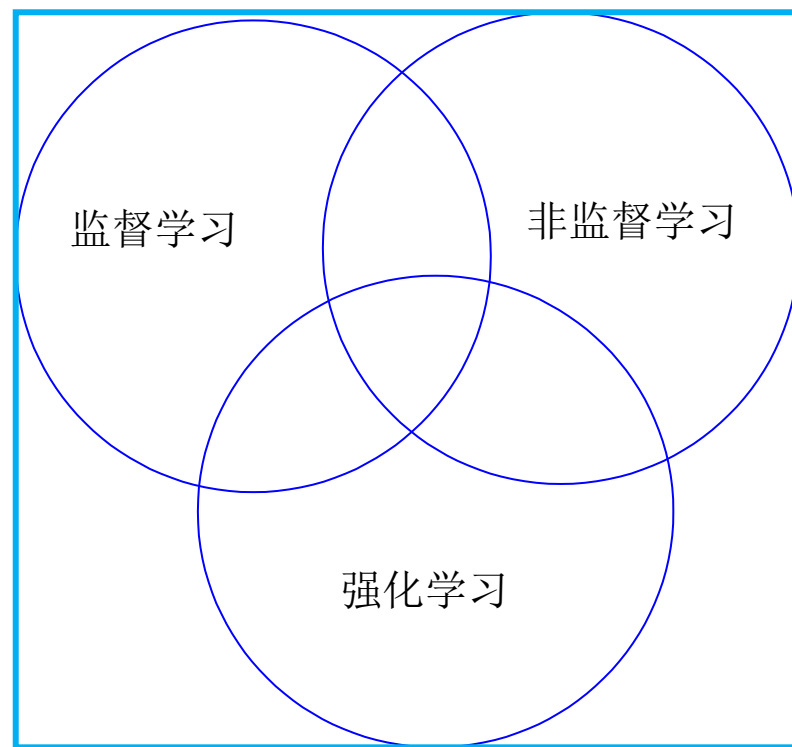
每天观察市场状态 -> 做出买/卖/持仓决定 -> 获得收益/亏损 -> 观察新的市场状态 -> 再决策...

本质上是一个序列决策问题，正好对应 RL 的框架。

什么是强化学习

强化学习 (Reinforcement Learning) :

- 机器学习的一个分支：监督学习、无监督学习、强化学习
- 强化学习的思路和人比较类似，是在实践中学习
- 比如学习走路，如果摔倒了，那么我们大脑后面会给一个负面的奖励值 => 这个走路姿势不好；如果后面正常走了一步，那么大脑会给一个正面的奖励值 => 这是一个好的走路姿势
- 与监督学习的区别，没有监督学习已经准备好的训练数据输出值，强化学习只有奖励值，但是这个奖励值和监督学习的输出值不一样，它不是事先给出的，而是延后给出的（比如走路摔倒）
- 与非监督学习的区别，在非监督学习中既没有输出值也没有奖励值的，只有数据特征，而强化学习有奖励值（为负是为惩罚），此外非监督学习与监督学习一样，数据之间也都是独立的，没有强化学习这样的前后依赖关系



机器学习

什么是强化学习

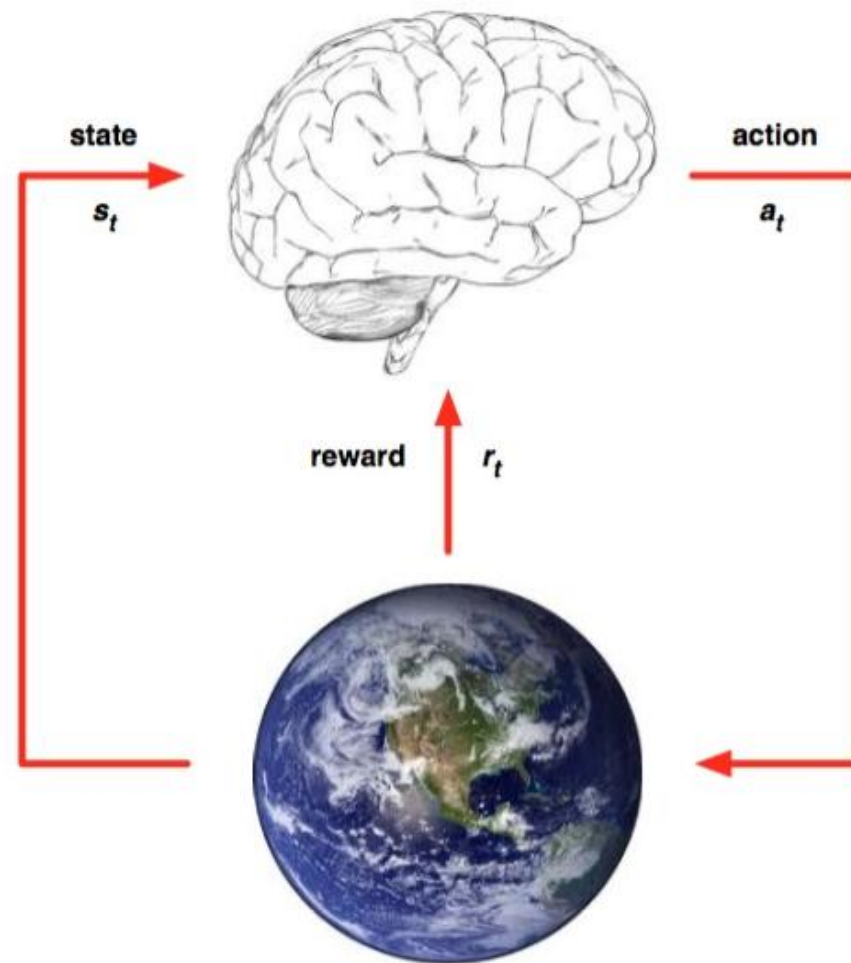
强化学习 (Reinforcement Learning) :

- 可以应用于不同领域：神经科学、心理学、计算机科学、工程领域、数学、经济学等

强化学习的特点：

- 没有监督数据、只有奖励信号
- 奖励信号不一定是实时的，很可能是延后的，甚至延后很多
- 时间（序列）是一个重要因素
- 当前的行为影响后续接收到的数据

强化学习有广泛的应用：游戏AI，推荐系统，机器人仿真，投资管理，发电站控制

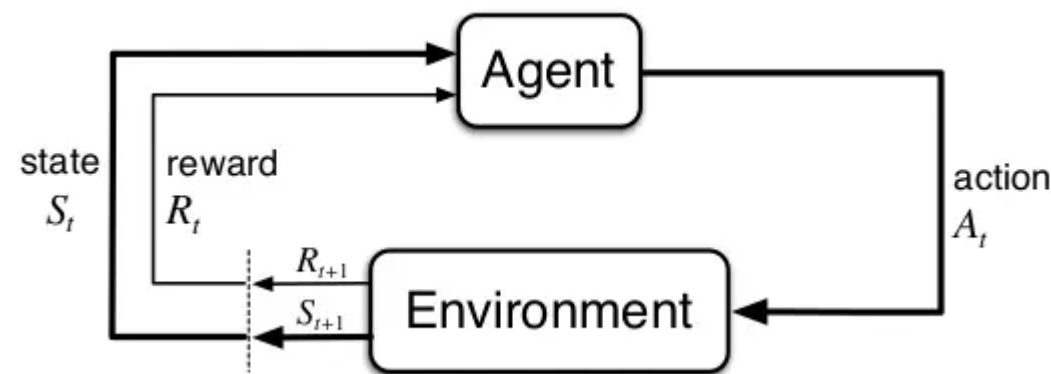


什么是强化学习

基本概念：

- 个体，Agent，学习器的角色，也称为智能体
- 环境，Environment，Agent之外一切组成的、与之交互的事物
- 动作，Action，Agent的行为
- 状态，State，Agent从环境获取的信息
- 奖励，Reward，环境对于动作的反馈
- 策略，Policy，Agent根据状态进行下一步动作的函数
- 状态转移概率，Agent做出动作后进入下一状态的概率

四个重要的要素：状态(state)、动作(action)、策略 (policy)、奖励(reward)

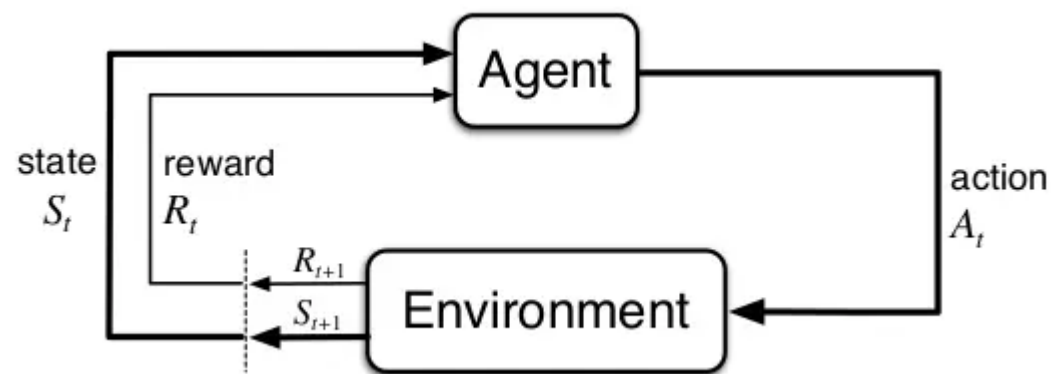


强化学习的基本结构

什么是强化学习

强化学习：

- RL考虑的是个体（Agent）与环境（Environment）的交互问题
- 目标是找到一个最优策略，使Agent获得尽可能多的来自环境的奖励
- 比如赛车游戏，游戏场景是环境，赛车是Agent，赛车的位置是状态，对赛车的操作是动作，怎样操作赛车是策略，比赛得分是奖励
- 很多情况下，Agent无法获取全部的环境信息，而是通过观察 (Observation)来表示环境（environment），也就是得到的是自身周围的信息



强化学习的基本结构

强化学习的基本概念

- 奖励 (Reward)

R_t 是信号的反馈，是一个标量，它反映个体在 t 时刻做得怎么样，个体的工作就是最大化累计奖励

强化学习假设是，所有问题解决都可以被描述成最大化累积奖励

- 序列决策 (Sequential Decision Making)

目标：选择一定的行为系列以最大化未来的总体奖励

这些行为可能是一个长期的序列

奖励可能而且通常是延迟的

有时候宁愿牺牲短期奖励，从而获取更多的长期奖励

- 个体与环境的交互 (Agent & Environment)

从个体的视角：

在 t 时刻，Agent 可以：

有一个对于环境的观察评估 Q_t

做出一个行为 A_t

从环境得到一个奖励信号 R_{t+1}

环境可以：

接收个体的动作 A_t

更新环境信息，同时使得个体可以得到下一个观测 Q_{t+1}

给个体一个奖励信号 R_{t+1}

CASE: 搭建RL交易环境

Gym与交易环境

要素	CartPole (倒立摆)	StockTradingEnv (股票交易)
State	cart位置/速度, pole角度/速度 (4维)	回看5日OHLC的Z-score (20维)
Action	向左/向右 (2个)	持仓/买入/卖出 (3个)
Reward	每步+1 (活着就有奖励)	收益率 (扣手续费)
终止条件	杆子倒下 / 超200步	数据走完 (773步)

CartPole是Gym官方的Hello World

=> 我们把股票交易包装成了同样的接口, 这样DQN算法可以直接用, 不需要改动

什么是gym

gym使用

- OpenAI开发的，用于强化学习算法研发的工具包
- 由两部分组成：

1) Gym开源库

提供了一些仿真环境（游戏）合集，包括了环境的交互，和公共接口，可以让用户设计自己的算法

- OpenAI Gym服务

提供一个站点和API（比如经典控制问题：CartPole-v0），允许用户对他们的测试结果进行比较



Project: cartpole AI

- 第一个gym游戏: CartPole (倒立摆)

```
import gym
```

```
env = gym.make('CartPole-v0')
```

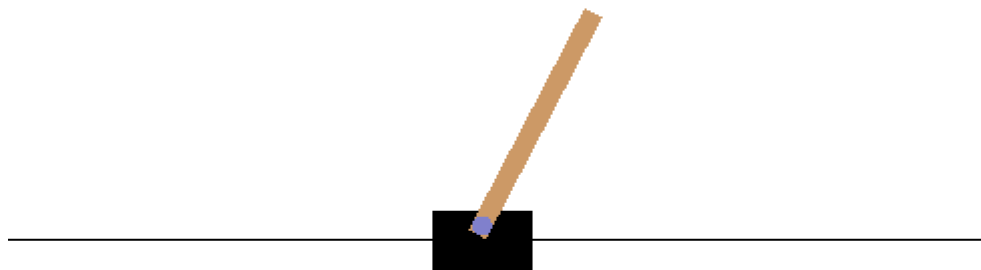
```
env.reset()
```

```
for _ in range(1000):
```

```
    env.render()
```

```
    env.step(env.action_space.sample())
```

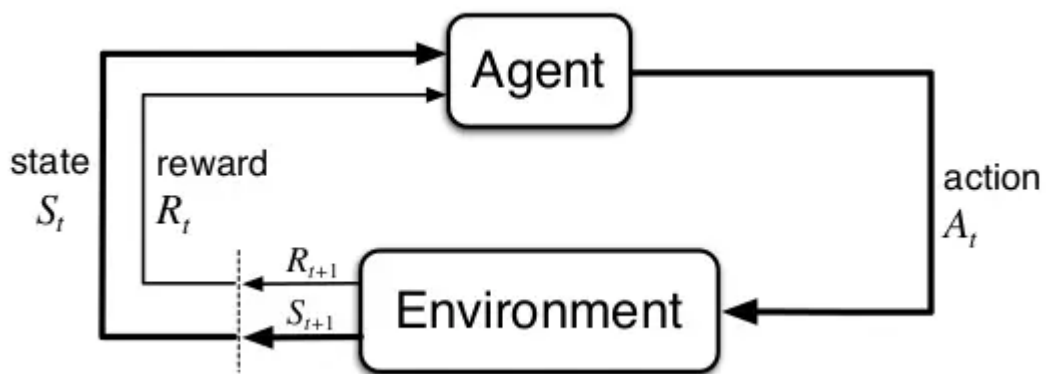
```
env.close()
```



Project: cartpole AI

强化学习的基本流程，即 agent-environment loop

在每个时间点上，智能体 (Agent) 选择一个动作 (action)，
环境返回上一次action的观测 (Observation) 和奖励 (Reward)



在Gym中，使用env.step() 进行每一步仿真，同时返回 4 个参数：

1) 观测 Observation (Object)

当前step执行后，环境的观测(类型为对象)

2) 奖励 Reward (Float)

执行动作(action)后，智能体(agent)获得的奖励，不同的环境中奖励值变化范围也不相同，强化学习的目标就是总奖励最大化

3) 完成 Done (Boolean)

当 Done=True 时，表明当前回合(episode) 结束，需要将环境重置 env.reset

4) 信息 Info (Dict)

针对调试过程的诊断信息。通常Agent仿真评估中不会使用info

Project: cartpole AI

倒立摆可以获取的信息:

- 倒立摆一共有两个Action:

- 1) Action = 0, 小车受到的力向左

- 2) Action = 1, 小车受到的力向右

- State, env返回的状态, 是4个数字的数组, 比如
[0.0113451, 0.01545243, 0.00934462, 0.02343784]

分别代表: cart位置和速度, pole的角度和速度

cart位置: $-2.4 \sim 2.4$

cart速度: $-\infty \sim \infty$

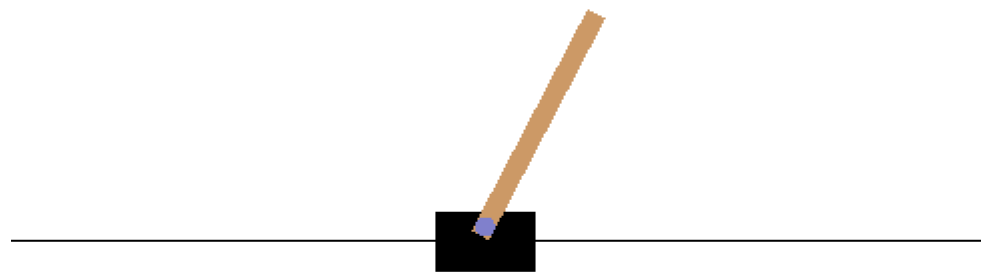
pole角度: $-0.5 \sim 0.5$ (radian)

pole角速度: $-\infty \sim \infty$

- Screen的图像

```
screen = env.render(mode='rgb_array').transpose((2,  
0, 1))
```

```
screen.shape # (3, 400, 600)
```



CASE：搭建RL交易环境

Gym 是强化学习（RL）领域最主流的标准环境接口库。

它定义了一套统一的 API，让 Agent（智能体）和 Environment（环境）之间可以标准化地交互。

核心交互循环很简单：

Agent 观察状态(obs) -> 选择动作(action) -> 环境执行 -> 返回(新状态, 奖励, 是否结束)

Gym 定义的核心 API:

- `env.reset()` -- 重置环境，返回初始观测
- `env.step(action)` -- 执行一个动作，返回 (obs, reward, terminated, truncated, info)
- `env.observation_space` -- 定义观测的形状和范围
- `env.action_space` -- 定义动作的类型和数量

把股票交易问题包装成一个 Gym 环境
=> 可以用 DQN、PPO 等RL 算法直接训练。

脚本逻辑

模块1: StockTradingEnv -- 交易环境 (核心)

继承 gymnasium.Env 的自定义环境，把股票择时交易建模为 RL 问题：

状态空间 (20维)：回看 5 天的 OHLC (开高低收)，每个价格用过去 252 天收盘价做 Z-score 标准化，即 $(price - mean) / std$

共 5天 x 4个价格 = 20维

动作空间 (离散3种)：

- 0 = 持仓不动 (hold)
- 1 = 买入 (全仓做多)
- 2 = 卖出 (平仓)

奖励设计 (4种情况)：

当前状态	动作	奖励
空仓	买入	下一日收益率 - 佣金
空仓	持有/卖出	0
持仓	继续持有	下一日收益率
持仓	卖出	持仓期间总收益 - 手续费

脚本逻辑

模块2: `_get_obs()` -- 观测构造

```
def _get_obs(self):
    """构造观测向量: lookback 日 OHLC 的 Z-score"""
    idx = self.current_step
    lb = self.lookback
    hist_start = max(0, idx - self.norm_window)
    hist_close = self.df['close'].iloc[hist_start:idx].values
    mu = hist_close.mean()
    sigma = hist_close.std()
    if sigma < 1e-8:
        sigma = 1.0
    obs = []
    for offset in range(lb, 0, -1):
        row = self.df.iloc[idx - offset]
```

```
        obs.extend([
            (row['open'] - mu) / sigma,
            (row['high'] - mu) / sigma,
            (row['low'] - mu) / sigma,
            (row['close'] - mu) / sigma,
        ])
    return np.array(obs, dtype=np.float32)
```

取过去 252 天收盘价算均值/标准差，然后把最近 5 天的 OHLC 做标准化。

=> 好处是消除价格绝对值的影响，让模型学到的是价格的相对模式。

脚本逻辑

模块3: `step()` -- 环境推进

RL 的核心循环单元。每调用一次：

- 根据当前持仓状态 + 动作，计算奖励
- 更新资金、持仓、净值
- 记录轨迹（用于后续可视化）
- 推进到下一天，判断是否结束

模块4: `run_random_agent()` -- 随机策略验证

用随机动作跑完整个 episode，目的是验证环境的正确性：

- API 调用是否正常
- 奖励计算是否合理
- episode 能否正常结束

从结果看，随机 Agent 的收益率在 -18% ~ +21% 之间波动，符合预期（随机操作不稳定）。

脚本逻辑

模块5: `plot_episode()` -- 可视化

绘制两张子图:

- 上图: 收盘价曲线 + 买卖点标记 (红色三角=买入, 绿色三角=卖出)
- 下图: 策略净值 vs 买入持有基准, 直观对比策略效果

脚本逻辑

整体执行流程：

- 1) 加载 510050.SH 数据 (1027天)
- 2) 创建 StockTradingEnv
- 3) 验证 API (reset/step)
- 4) 随机Agent跑3轮 episode
- 5) 可视化最后一轮交易轨迹

这是整个 RL 交易系列的基础设施，即先把股票交易翻译成 Gymnasium 能理解的语言（状态/动作/奖励），后续我们可以在此基础上接入 DQN、回测、智能拆单等更高级的 RL 算法。

CASE：搭建RL交易环境

Thinking：为什么要自己构建环境？

Gym 自带的环境（CartPole、Atari）是游戏环境，不是交易环境。

=> 我们需要构建一个股票交易环境，让 RL Agent 可以在真实数据上学习。

环境核心设计：

```
# 和 CartPole 一样的 Gymnasium 接口
env = StockTradingEnv(df, lookback=5, norm_window=252)
obs, info = env.reset()      # 重置环境
obs, reward, terminated, truncated, info = env.step(action) # 执行动作
```

要素	CartPole (倒立摆)	StockTradingEnv (股票交易)
State	cart位置/速度, pole角度/速度 (4维)	回看5日 OHLC 的 Z-score (20维)
Action	向左 / 向右 (2个)	持仓 / 买入 / 卖出 (3个)
Reward	每步 +1 (活着就有奖励)	收益率 (扣手续费)
终止条件	杆子倒下	数据走完

CASE: 搭建RL交易环境

状态空间: Z-score 标准化

```
# 取过去252日收盘价计算均值和标准差
mu = hist_close.mean()
sigma = hist_close.std()

# 对每个 OHLC 值做 Z-score: (x - mu) / sigma
obs = [(open-mu)/sigma, (high-mu)/sigma,
        (low-mu)/sigma, (close-mu)/sigma] # x 5天 = 20维
```

Thinking: 为什么用 Z-score 而不是原始价格?

茅台价格 ~1800 元, ETF 价格 ~3 元。

如果用原始价格, 模型学到的是: 3块钱附近要买入 (这种无意义的规律)

Z-score 让模型学到的是: 相对位置 (价格偏离均值多少个标准差)

CASE：搭建RL交易环境

奖励函数: 四种情况

当前状态	动作	奖励	含义
空仓	买入	下一日收益率 - 佣金	买入后获得多头收益，扣手续费
空仓	持有/卖出	0	不操作无收益
持仓	持有/买入	下一日收益率	继续持有获得日收益
持仓	卖出	持仓期收益 - 佣金 - 印花税	平仓兑现收益，扣全部费用

交易成本: 佣金万1.5 (买卖各收) + 印花税千0.5 (仅卖出)。每次完整交易（买+卖）成本约 0.08%。

CASE: 搭建RL交易环境

环境验证: 随机 Agent 运行结果

运行结果 (1-搭建RL交易环境.py):

数据: 510050.SH (上证50ETF) | 1027个交易日 | 2022-01-04 ~ 2026-04-09

价格范围: 2.100 ~ 3.240

有效交易区间: 第252天 ~ 第1025天 (共773步)

随机 Agent 验证:

Episode 1: 773步 | 累计奖励: -0.0467 | 最终净值: 859,511 | 收益率: -14.05%

Episode 2: 773步 | 累计奖励: -0.0240 | 最终净值: 885,271 | 收益率: -11.47%

Episode 3: 773步 | 累计奖励: -0.2416 | 最终净值: 803,441 | 收益率: -19.66%

随机交易稳定亏损 10%~20% -- 说明环境逻辑正确: 随机交易 + 手续费 = 必然亏损。

打卡：搭建RL交易环境



以 510050.SH (上证50ETF) 为例，使用 Gyn 构建RL交易环境：

- 构建 StockTradingEnv 自定义交易环境
- 状态空间设计: 回看5日OHLC，用过去252日收盘价做 Z-score 标准化，得到20维观测向量
- 动作空间设计: 离散3个动作 -- 持仓(0)、买入(1)、卖出(2)
- 奖励函数设计: 4种情况(空仓买入/空仓持有/持仓持有/持仓卖出)
- 用随机Agent跑3轮完整Episode，验证环境的API调用、奖励计算、终止条件是否正常

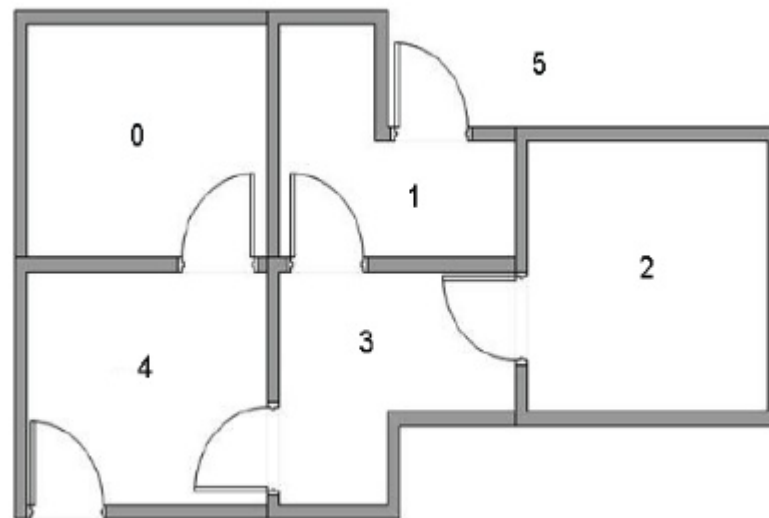
从 Q-Learning 到 DQN

Q-Learning原理

Project: 迷宫问题

Project: 迷宫问题

- 迷宫为5个房间（实际可以为N个房间），房间与房间之间通过门连接，编号0到4,5号是房子外边，即我们想要达到的终点
- 将AI随机放在任一房间内，如何找到终点的路径

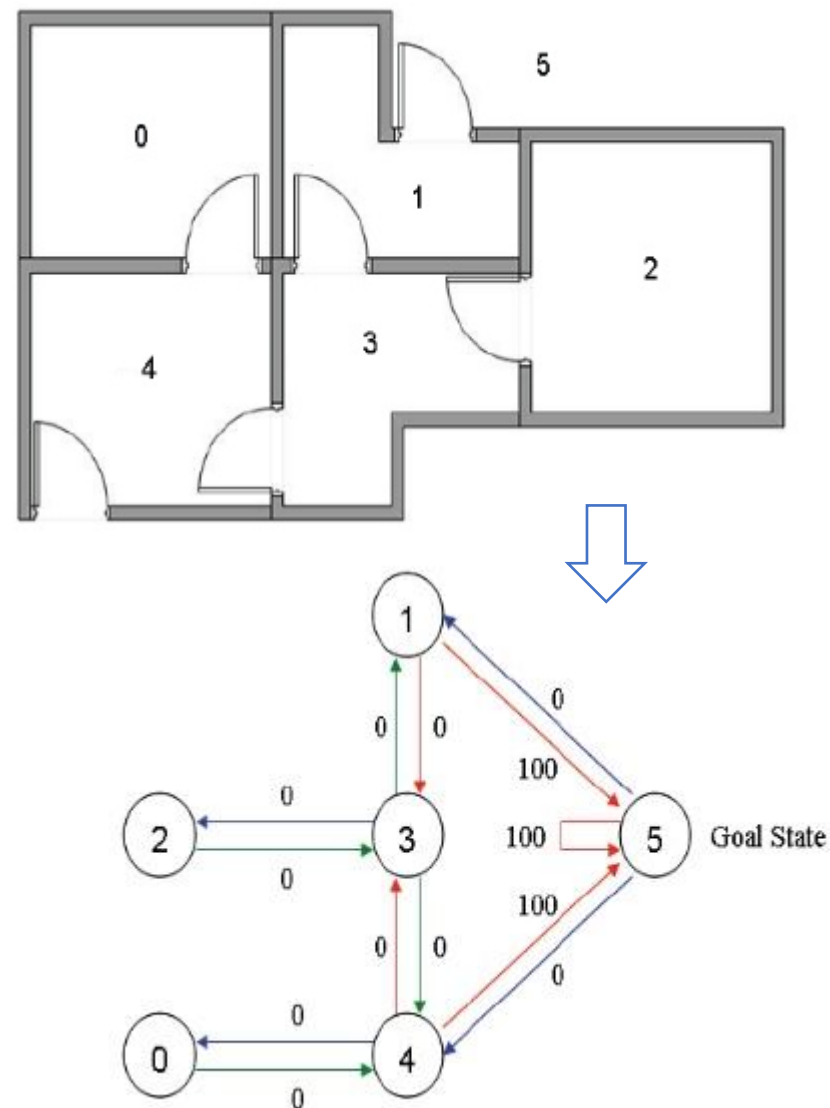


Project: 迷宫问题

QLearning:

- Q函数, 也称为动作值函数, 有两个输入: 「状态」和「动作」, 它将返回在这个状态下执行该动作的未来奖励期望
- 将agent随机放在任一房间内, 每打开一个房门返回一个 reward
- 根据房间之间连通性的关系, 可以得到Reward矩阵 (R矩阵)
- 可以把 Q 函数视为一个在 Q-table 上滚动的读取器

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100



Project: 迷宫问题

迷宫 = 地图+路线评分表

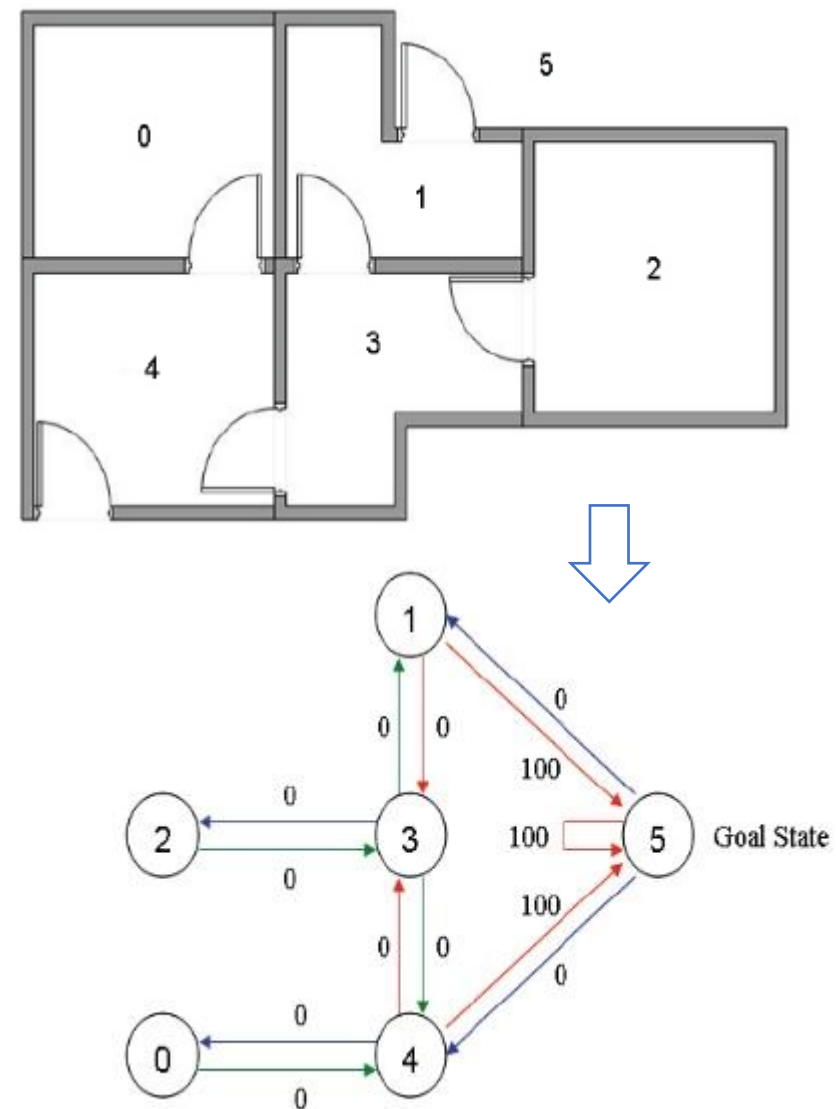
想象你有一个导航APP:

5个房间是5个地点, 房间之间的门是路线

Q表就是每条路线的评分表

=> 目标是找到从任意房间到出口(5号)的最优路线。

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100



Project: 迷宫问题

Q-learning是典型的基于价值 (Value) 函数的强化学习方法

其中Q是一个数值 (即价值value) , 在初始化时可能被赋予一个任意数值

在迭代时刻t, 我们有状态 s_t 此时代理做出动作 a_t , 然后得到奖励 r_t , 从而进入到一个更新的状态 s_{t+1} ,

然后对Q值进行更新, 更新公式为

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learnig rate}} \cdot \overbrace{\left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}^{\text{learned value}}$$

新评分 = 走这条路的即时收益 + 折扣 × 到了下一个房间后能拿到的最高评分

Project: 迷宫问题

使用QLearning:

- 初始化Q表, 用于记录状态-动作对的值, 每个episode中的每一步都会根据公式更新一次Q表

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

为了简便, 将学习率 α 设为1, 更新公式为:

$$Q(S_t, A_t) \leftarrow R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$$

当前状态s的最大将来奖励等于下一状态s' 的最大将来奖励乘以折扣因子

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

until S is terminal

Project: 迷宫问题

```
# 模拟迷宫问题
```

```
GAMMA = 0.8
```

```
Q = np.zeros((6,6))
```

```
R=np.asarray([[[-1,-1,-1,-1,0,-1],
```

```
[-1,-1,-1,0,-1,100],
```

```
[-1,-1,-1,0,-1,-1],
```

```
[-1,0, 0, -1,0,-1],
```

```
[0,-1,-1,0,-1,100],
```

```
[-1,0,-1,-1,0,100]]])
```

```
# 取每一行的最大值
```

```
def getMaxQ(state):
```

```
# 通过选取最大动作值来进行最优策略学习
```

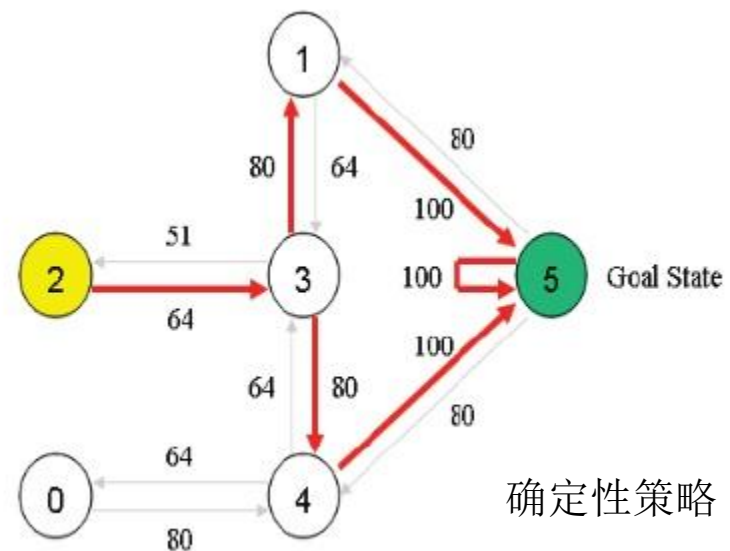
```
return max(Q[state, :])
```

State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

Project: 迷宫问题

```
def QLearning(state):  
    curAction = None  
    for action in range(6):  
        if(R[state][action] == -1):  
            Q[state, action] = 0  
        else:  
            curAction = action  
            Q[state, action] = R[state][action] + GAMMA *  
getMaxQ(curAction)  
# 模拟1000次  
for count in range(1000):  
    for i in range(6):  
        QLearning(i)  
# 显示保留小数点后一位  
np.set_printoptions(precision=1)  
print(Q/5)
```

```
[[ 0.  0.  0.  0. 80.  0.]  
 [ 0.  0.  0. 64.  0. 100.]  
 [ 0.  0.  0. 64.  0.  0.]  
 [ 0. 80. 51  0. 80.  0.]  
 [64.  0.  0. 64.  0. 100.]  
 [ 0. 80.  0.  0. 80. 100.]]
```



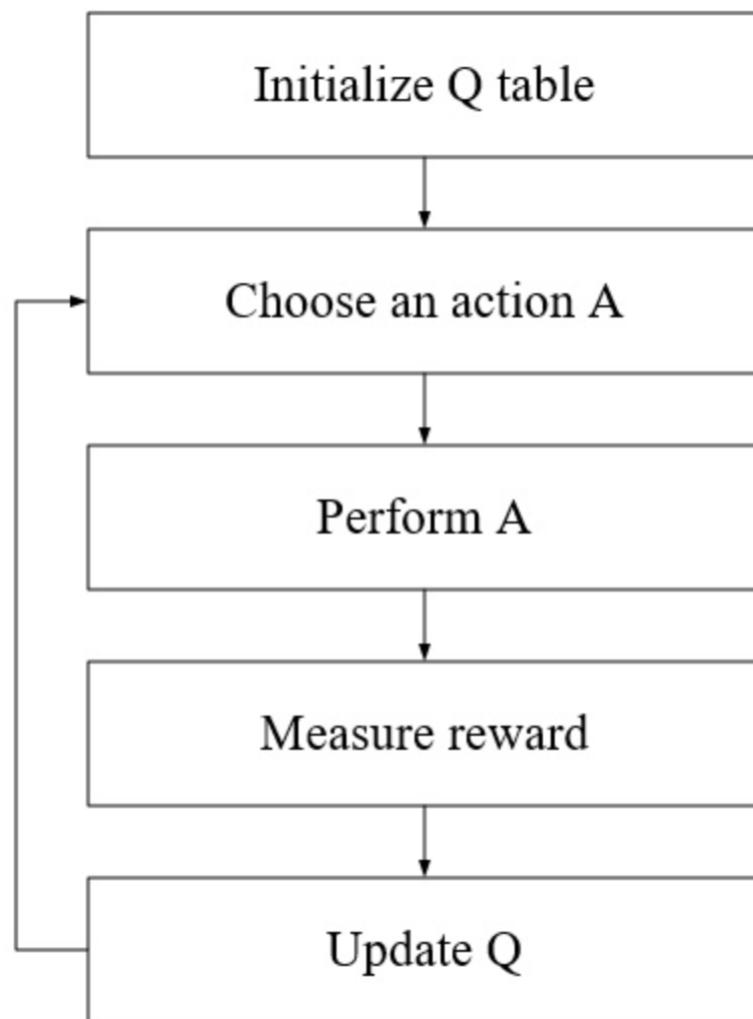
QLearning

Qlearning:

- 一个(s, a)对应一个Q值, 可以把Q值看作一个很大的表格

横列代表s, 纵列代表a, 数值为Q值

	a1	a2	a3	a4
s1	Q(1,1)	Q(1,2)	Q(1,3)	Q(1,4)
s2	Q(2,1)	Q(2,2)	Q(2,3)	Q(2,4)
s3	Q(3,1)	Q(3,2)	Q(3,3)	Q(3,4)
s4	Q(4,1)	Q(4,2)	Q(4,3)	Q(4,4)



Summary (Qlearning的学习过程)

Step 1: 定义环境 -- R矩阵(奖励矩阵)

```
# R矩阵: 6x6, R[i][j] 表示从房间i走到房间j的即时奖励
# -1 = 此路不通, 0 = 可以走, 100 = 到达终点(大奖励)
R = np.asarray([
    [-1, -1, -1, -1, 0, -1], # 房间0: 只能去房间4
    [-1, -1, -1, 0, -1, 100], # 房间1: 可以去房间3或5(出口)
    [-1, -1, -1, 0, -1, -1], # 房间2: 只能去房间3
    [-1, 0, 0, -1, 0, -1], # 房间3: 可以去房间1/2/4
    [0, -1, -1, 0, -1, 100], # 房间4: 可以去房间0/3或5(出口)
    [-1, 0, -1, -1, 0, 100] # 房间5: 终点(自环)
])
```

Step 2: 学习过程 -- 更新Q表

```
# Q表: 初始全0, 学习后每个格子存的是"长期价值"
Q = np.zeros((6, 6))
GAMMA = 0.8 # 折扣因子: 越接近1越重视未来收益

# 更新公式 (简化版, 学习率=1):
# Q(当前房间, 走的门) = R(当前房间, 走的门) + 0.8 *
# max(Q[下一个房间, :])

Q[state, action] = R[state, action] + GAMMA *
max(Q[next_state, :])
```

QLearning在股票交易中的应用

Thinking: QLearning 如何应用在股票交易中?

Q 函数 $Q(s, a)$ 表示: 在状态 s 下执行动作 a , 未来能获得的总奖励期望。

Q 表格示例

	买入	卖出	持仓	
状态1 (低位)	+1.5	-0.3	+0.2	<-- 在低位时, 买入价值最高
状态2 (高位)	-0.5	+1.2	+0.1	<-- 在高位时, 卖出价值最高
状态3 (震荡)	+0.1	-0.2	+0.8	<-- 震荡时, 持仓价值最高

Q 值更新公式

Q 函数 $Q(s, a)$ 表示：在状态 s 下执行动作 a ，未来能获得的总奖励期望。

$$Q(s, a) \leftarrow Q(s, a) + \alpha * [R + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$$



新Q值 = 旧Q值 + 学习率 * [即时奖励 + 折扣 * 下一状态最优Q值 - 旧Q值]

Thinking: Q-Table 的局限性?

股票的状态是连续值（价格 2.100~3.240、成交量、各种技术指标）

=> 状态空间几乎无穷大，表格根本存不下。怎么办？

DQN原理

基于DQN的强化学习

Thinking: QTable的局限性, 使用表格来表示 $Q(s, a)$, 但是现实问题中状态太多, 使用表格根本存不下, 怎么办?

	a1	a2	a3	a4
s1	$Q(1,1)$	$Q(1,2)$	$Q(1,3)$	$Q(1,4)$
s2	$Q(2,1)$	$Q(2,2)$	$Q(2,3)$	$Q(2,4)$
s3	$Q(3,1)$	$Q(3,2)$	$Q(3,3)$	$Q(3,4)$
s4	$Q(4,1)$	$Q(4,2)$	$Q(4,3)$	$Q(4,4)$

迷宫 (Q表够用)

- 6个房间 (离散状态)
- 6个动作
- Q表: $6 \times 6 = 36$ 格



股票交易 (Q表存不下)

- 价格、成交量、MACD... (连续值)
- 20维状态, 每维100个取值
- Q表: $100^{20} = 10^{40}$ 格 -- 存不下

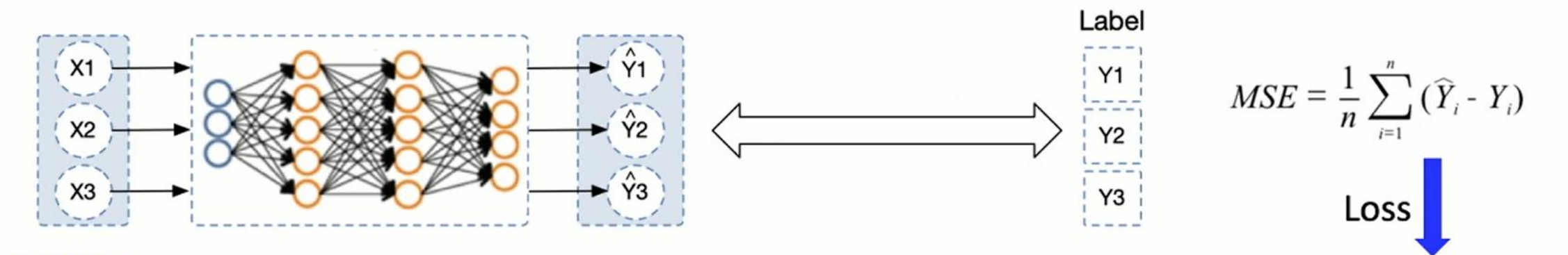
用一个函数来代替表格:

给函数输入状态, 它直接算出每个动作的Q值, 不用查表了。

=> 这个函数就是神经网络 DQN。

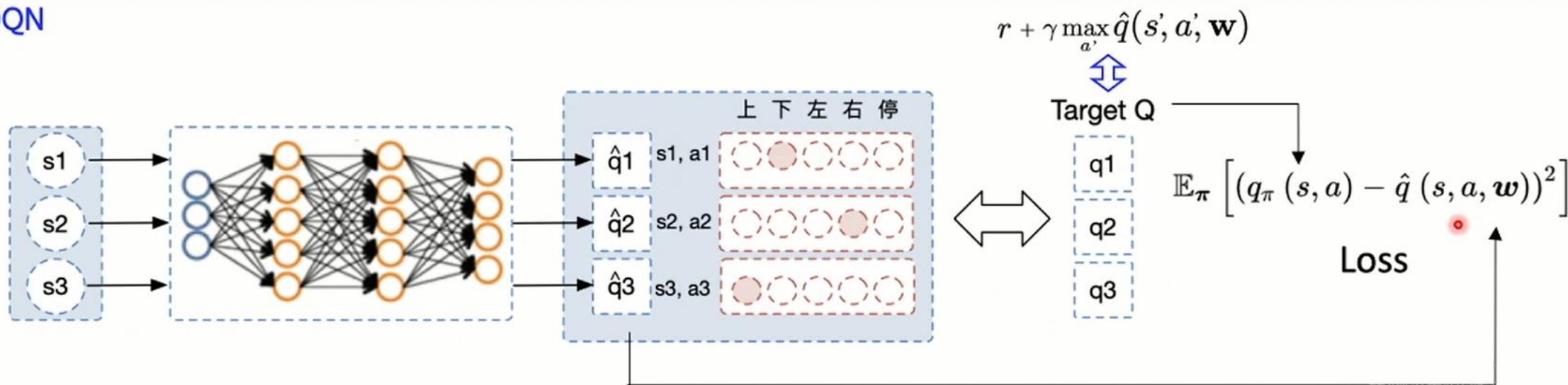
基于DQN的强化学习

DQN = Qlearning + 神经网络



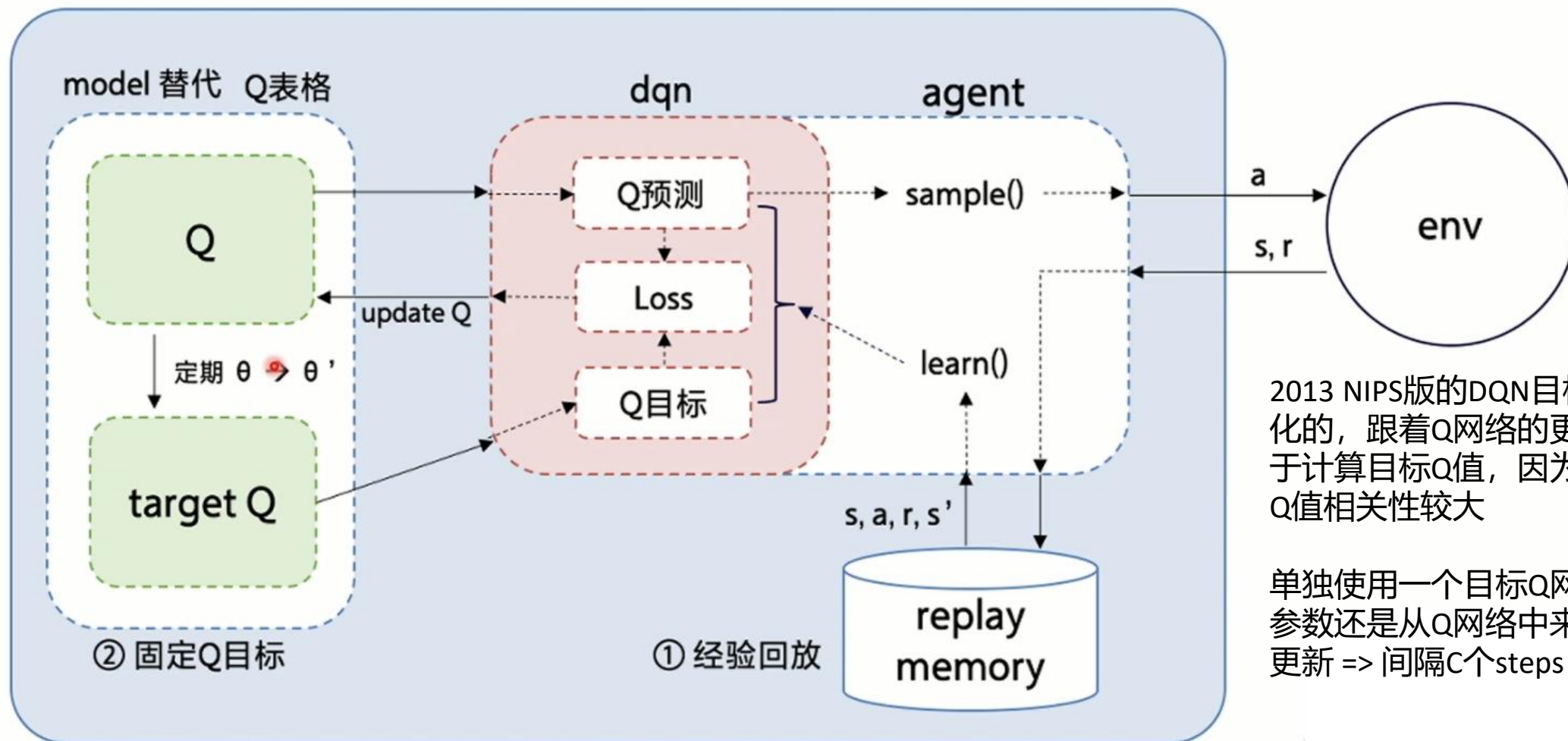
监督学习

DQN



基于DQN的强化学习

DQN流程



2013 NIPS版的DQN目标Q网络是动态变化的，跟着Q网络的更新而变化 => 不利于计算目标Q值，因为目标Q值和当前的Q值相关性较大

单独使用一个目标Q网络，目标Q网络的参数还是从Q网络中来，只不过是延迟更新 => 间隔C个steps，更新目标Q网络

DQN: 用神经网络替代 Q 表格

核心思想: 用一个神经网络 $f(s, w)$ 来近似 Q 函数, 输入状态 s , 输出每个动作的 Q 值。

DQN 网络结构

输入: 市场状态 (20维向量)

|

|-- Linear(20 -> 64) + ReLU # 第一层: 把20维信息压缩提取

|-- Linear(64 -> 64) + ReLU # 第二层: 进一步提取特征

|-- Linear(64 -> 3) # 第三层: 输出3个Q值

|

输出: [Q(持仓)=0.2, Q(买入)=1.5, Q(卖出)=-0.3]

--> 选最大的: 买入!

```
class QNetwork(nn.Module):  
    def __init__(self, state_dim=20, action_dim=3):  
        self.net = nn.Sequential(  
            nn.Linear(20, 64), # 输入20维状态  
            nn.ReLU(),  
            nn.Linear(64, 64), # 中间提取特征  
            nn.ReLU(),  
            nn.Linear(64, 3), # 输出3个动作的Q值  
        )
```

总参数量只有 5,699个 => 非常轻量

DQN怎么训练：错题本机制

Thinking: 现在有了神经网络, 但怎么训练它?

DQN的训练方式很像我们复习考试 —— 用一个错题本。

- 为什么需要错题本?

如果Agent今天买了股票赚了, 明天又买又赚, 后天还买... 它只记住了"买买买", 忘了之前亏钱的教训。

=> 这就像只复习最近做的题, 前面的题全忘了。

- 问题: 经验有时间连续性

今天的状态受昨天的影响, 昨天受前天的影响... 样本之间不独立。

直接按时间顺序训练, 网络容易"只记住最近的模式", 忘掉早期的规律。

- 解决: 经验回放 (Experience Replay)

把每次交易经历 (当时状态, 做了什么, 赚了多少钱, 下一个状态) 存进一个错题本。

训练时随机抽取64条经验来学习, 打乱时间顺序, 避免只记最近的。

DQN怎么训练：错题本机制

经验回放: 存和取

存: 每做一步交易, 把经验存起来

buffer.push(state, action, reward, next_state, done)

错题本最多存10,000条, 满了就把最早的挤掉

取: 训练时随机抽64条

batch = random.sample(buffer, batch_size=64)

可能抽到3个月前的经验和昨天的经验混在一起

这样网络既能记住老的规律, 也能学到新的

错题本 (ReplayBuffer, 容量10000):

(状态_1, 买入, +0.02, 状态_2, False) <-- 3个月前的经验

(状态_2, 持有, +0.01, 状态_3, False)

...

(状态_8888, 卖出, -0.005, 状态_8889, False) <-- 最近的经验

=> random.sample(buffer, 64) 随机抽64条

=> 训练 mini-batch: 打乱了时间顺序, 网络不会只记住最近的模式

DQN怎么训练：老师-学生 双网络

Thinking: 有了错题本还不够, 还有个问题: 训练的时候, 谁来出标准答案?

- 问题: 自己给自己出题, 答案一直在变

DQN的训练本质是一个回归问题: 网络预测Q值, 要跟一个标准答案 (目标Q值)对比

=> 然后调整参数。 (但标准答案也是这个网络自己算出来的)

- Moving Target (移动靶) 问题

想象一个学生在做模拟考试, 但出题的老师 and 答题的学生是同一个人。

每次学生提高了一点, 老师 (同一个网络)的标准答案也跟着变了。

=> 学生永远追不上标准答案, 训练无法收敛。

DQN怎么训练：老师-学生 双网络

解决方法: 目标网络 (Target Network)

用两个一模一样的网络:

- 学生网络 (Q网络): 每步都在学习更新参数
- 老师网络 (Target网络): 参数冻结, 每隔100步才从学生那里同步一次

这样老师的标准答案在100步内是固定的, 学生有一个稳定的学习目标。

DQN怎么训练：老师-学生 双网络

解决方法: 目标网络 (Target Network)

学生网络 (Q网络)

- 参数: 每步更新
- 职责: 预测Q值



预测: $Q(s, a) = 1.5$

老师网络 (Target网络)

- 参数: 暂时冻结
- 职责: 出标准答案



标准答案: $r + 0.99 * \max Q = 1.8$

差距 = $1.8 - 1.5 = 0.3$



调整学生网络参数, 让预测更接近标准答案



每隔100步: 学生参数 --> 复制给老师 (老师也进步了)

DQN怎么训练：老师-学生 双网络

```
# 学生网络预测: 当前状态下买入的Q值
current_q = q_net(states).gather(1, actions)

# 老师网络出标准答案: 奖励 + 折扣 * 下一状态的最优Q值
with torch.no_grad():
    next_q = target_net(next_states).max(dim=1)[0]
    target_q = rewards + 0.99 * next_q * (1 - dones)

# 让预测接近标准答案 (MSE损失)
loss = MSELoss(current_q, target_q)

# 每100步, 把学生参数同步给老师
if train_step % 100 == 0:
    target_net.load_state_dict(q_net.state_dict())
```

2-DQN择时策略.py

训练核心逻辑

DQN训练全景图

把两个机制放在一起, DQN的完整训练流程:

Step1. Agent 观察市场状态 (20维Z-score向量)

Step2. Epsilon-Greedy 选动作

90%概率: 用Q网络选Q值最大的动作

10%概率: 随机选 (探索新策略)

Step3. 在环境中执行动作, 获得奖励和新状态

Step4. 把 (状态, 动作, 奖励, 新状态) 存入错题本

Step5. 从错题本随机抽64条, 训练Q网络

Q网络预测 vs Target网络的标准答案 --> 计算差距

反向传播, 调整Q网络参数

Step6. 每100步: Q网络参数 --> 同步给Target网络

Step7. 衰减探索率: 1.0 --> 0.01 (越来越少随机, 越来越多利用)

=> 重复500个Episode (每个Episode遍历完整个交易区间)

Epsilon-Greedy 探索策略

刚开始训练时, Q网络什么都不懂, 所以让它90%的时间随机探索($\epsilon=1.0$)。

随着训练进行, 网络越来越准, 就逐步减少随机(ϵ 衰减到0.01)。

=> 新员工多试错, 老员工按经验做事。

Summary

DQN 三个关键技巧:

1. 经验回放

解决: 样本有时间连续性

方法: 存错题本, 随机抽样

=> 从回忆中学习

2. 目标网络

解决: 训练目标一直在变

方法: 双网络, 老师参数慢更新

=> 老师出题要稳定

3. Epsilon-Greedy

解决: 过早陷入局部最优

方法: 一定概率随机探索

=> 新人多试错

Thinking: 为什么这三个设计缺一不可?

- 去掉经验回放 => 网络只记住最近的行情, 换个市场环境就不行
- 去掉目标网络 => 训练不收敛, Loss震荡不下降
- 去掉探索 => 可能一开始碰巧学到一直不操作, 就永远不尝试买卖了

CASE: DQN择时策略

CASE: DQN择时策略

训练流程核心代码

```
for ep in range(1, 500 + 1):
    obs, _ = env.reset()
    while True:
        # 1. Epsilon-Greedy 选动作
        action = agent.select_action(obs, training=True)

        # 2. 执行动作, 获得环境反馈
        next_obs, reward, terminated, truncated, info = env.step(action)

        # 3. 存储经验到回放池 (s, a, r, s', done)
        agent.store_transition(obs, action, reward, next_obs, done)

        # 4. 从回放池随机采样 mini-batch 训练 Q 网络
        agent.train()

        # 内部: loss = MSE(Q(s,a), r + gamma * max Q_target(s',a'))
```

```
obs = next_obs
if terminated: break

# 5. 衰减探索率: 1.0 -> 0.01
agent.decay_epsilon()
```

2-DQN择时策略.py
(train_dqn 函数)

CASE: DQN择时策略

超参数配置:

- Q网络: 20 -> 64 -> 64 -> 3 => 3层MLP, 仅 5,699 个参数
- gamma: 0.99 => 折扣因子: 重视长期收益
- lr: 1e-3 => 学习率
- epsilon: 1.0 -> 0.01 (decay=0.995) => 探索率: 指数衰减
- batch_size: 64 => 每次训练采样 64 条经验
- buffer_size: 10,000 => 经验回放池容量
- tau: 0.005 => 目标网络软更新系数

CASE: DQN择时策略

训练过程 (510050.SH, 500 Episodes, 约29分钟)

训练集: 839个交易日 | 2022-01-04 ~ 2025-06-30

Episode	Reward	Epsilon	NAV	Return	

1	-0.3090	0.9950	795,772	-20.42%	<-- 随机探索, 大亏
50	+0.7835	0.7783	1,335,725	+33.57%	<-- 开始学到东西
100	+0.8631	0.6058	1,397,416	+39.74%	
200	+1.0489	0.3670	1,524,600	+52.46%	
300	+1.6916	0.2223	2,134,686	+113.47%	<-- 训练集上表现最好
500	+1.5767	0.0816	2,010,301	+101.03%	<-- 稳定在高水平



可以看到 DQN 从 Episode 1 的 -20% 逐步学习到 Episode 300 的 +113%。
训练集上的高收益是正常的（有过拟合），
关键看样本外表现。

总训练步数: 292,437
后100轮平均奖励: 1.6530
最佳奖励: 2.1262

打卡：搭建DQN择时策略



以 StockTradingEnv 为训练环境，构建 DQN Agent

- 设置Q网络结构，比如 20->64->64->3
- 训练500轮 Episode

使用三个关键技巧: 经验回放(buffer=10000)、目标网络(每100步同步)、Epsilon-Greedy(1.0衰减到0.01)

- 训练过程中观察 Reward 和 Loss，以及收益率的变化

CASE：策略回测与评估

CASE：策略回测与评估

Thinking：我们还需要做样本外回测，为什么要做样本外回测？

训练集上 +113% 说明不了问题，因为模型可能只是记住了历史数据。

=> 必须在从未见过的数据上测试，才能验证策略的 真实效果。

在模型从未见过的数据上回测 DQN 策略，验证策略的真实效果。

CASE：策略回测与评估

脚本逻辑：

加载模型 --> 构建测试环境 --> epsilon=0 回测 --> 计算绩效指标 --> 对比买入持有

模块	功能	关键点
backtest()	回测引擎	调用 agent.select_action(obs, training=False)，即 epsilon=0，不再探索，纯利用学到的策略
calc_metrics()	绩效指标计算	计算年化收益、夏普比率(无风险利率2%)、最大回撤、卡玛比率、日胜率、盈亏比
plot_backtest_result()	回测可视化	三合一图: 价格+持仓信号、净值对比(DQN vs 买入持有)、回撤曲线
plot_trade_analysis()	交易分析	动作分布、买卖价格分布、持仓时长分布

- 训练数据: 2022-01-04 ~ 2025-06-30 (脚本2用这段数据训练)
- 测试数据: 2025-07-01 ~ 2026-04-09 (脚本3用这段模型从未见过的数据回测)
- 加载起点: 2024-01-01 (因为Z-score需要252天历史数据预热)

CASE：策略回测与评估

运行结果：

[1] 加载测试数据: 510050.SH

数据量: 543 个交易日

测试区间: 2025-07-01 ~ 2026-04-09

[2] 创建测试环境

[3] 加载 DQN 模型

模型已加载: models/dqn_best.pth

[4] 执行回测

总交易步数: 289

买入次数: 105

卖出次数: 161

[5] 绩效评估

--- DQN 择时策略 ---

总收益: +13.75%

年化收益: +11.89%

年化波动: 9.51%

夏普比率: 0.9886

最大回撤: 10.28%

卡玛比率: 1.1568

日胜率: 20.1%

盈亏比: 2.70

--- 买入持有基准 ---

总收益: +14.62%

年化收益: +12.64%

最大回撤: 11.42%

夏普比率: 0.7843

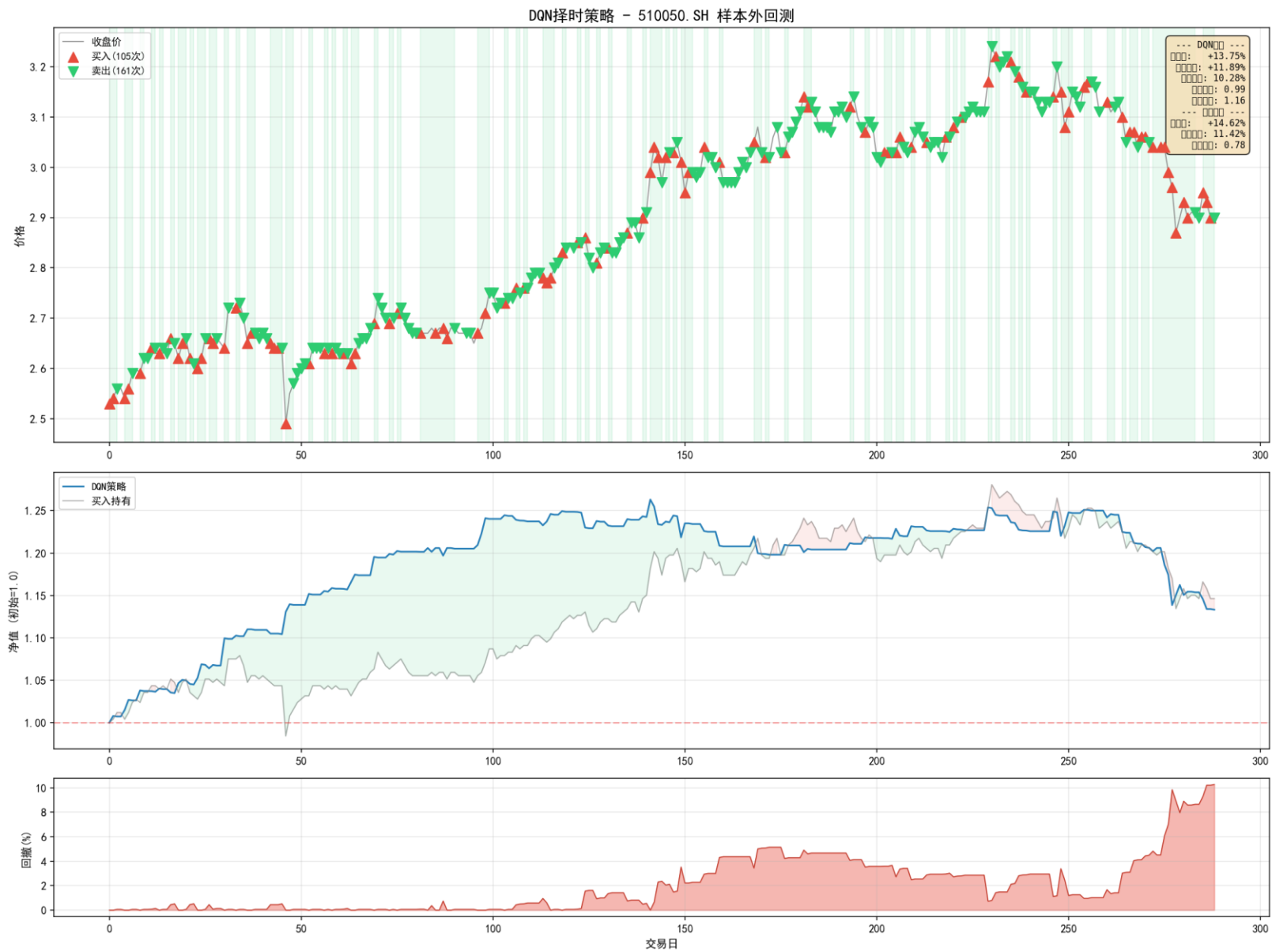
>>> 年化超额收益: -0.75%

[6] 可视化

图表已保存: outputs/3-DQN样本外回测.png

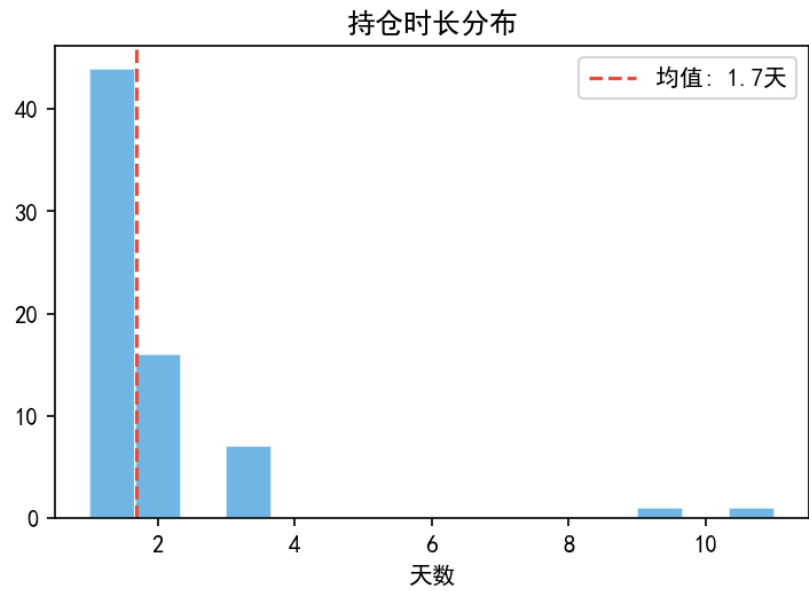
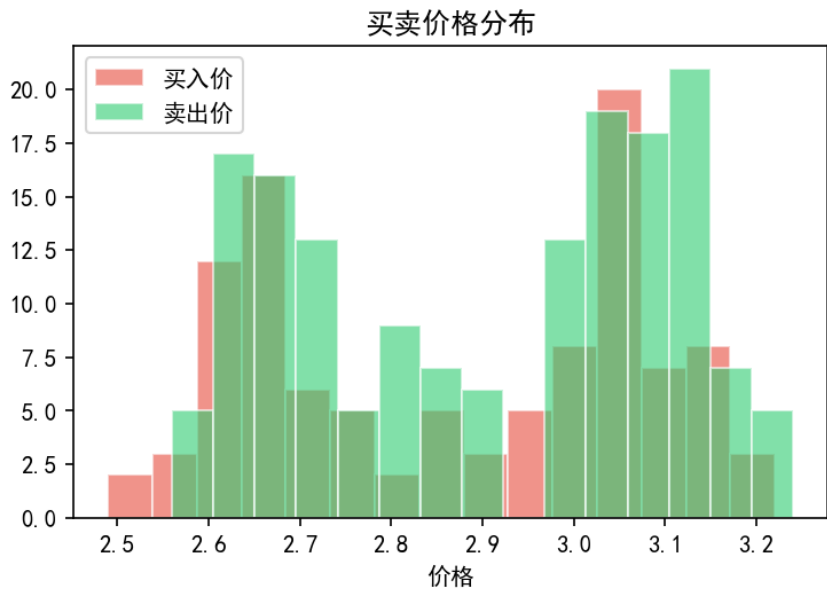
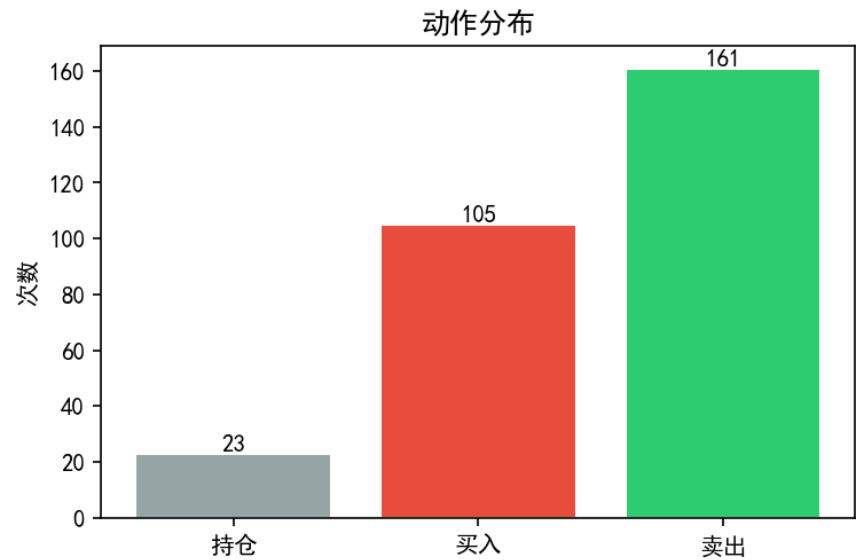
图表已保存: outputs/3-DQN交易分析.png

CASE: 策略回测与评估



CASE：策略回测与评估

DQN策略交易分析



CASE：策略回测与评估

DQN VS 买入持有：

指标	DQN 择时	买入持有	谁赢？
总收益	+13.75%	+14.62%	买入持有 (高0.87%)
夏普比率	0.9886	0.7843	DQN (+26%)
最大回撤	10.28%	11.42%	DQN (少1.14%)
卡玛比率	1.1568	1.11	DQN
盈亏比	2.70	--	赢的时候赚的多，亏的时候亏的少

Thinking: DQN 收益低于买入持有，是不是说明 RL 没用？

不是。要看具体场景：

- 1. 夏普比率更高 (0.99 vs 0.78) -- 每承担一单位风险，获得的回报更多
- 2. 最大回撤更小 (10.28 % vs 11.42%) -- 对于管理真金白银的人来说，回撤控制是核心
- 3. 本次测试恰好是上涨市 -- 单边牛市中买入持有天然占优。在震荡/下跌市中，DQN 的优势会更明显
- 4. RL 择时的核心价值是风控优于收益增长 -- 这是专业量化机构最看重的

打卡：策略回测与评估



加载脚本2训练好的 DQN 模型(dqn_best.pth), 进行回测:

- 在样本外区间 2025-07-01 ~ 2026-04-09 回测
- 观察回测结果

epsilon=0(纯利用, 不探索)的情况, 以及 DQN与买入持有的对比, 比如

DQN策略: 总收益+13.75%, 夏普比率0.99, 最大回撤10.28%, 盈亏比2.70

买入持有: 总收益+14.62%, 夏普比率0.78, 最大回撤11.42%

智能拆单

大单怎么买？

Ethan 通过 DQN 择时策略决定今天应该买入上证50ETF。但他需要买入 10万手（1000万股，约 3000 万元），这在市场上是一笔大单。

Thinking：一次性下单会怎样？

- Ethan 挂买单 10 万手 -> 卖盘的前几档很快被吃光
- 价格被他自己的买单推高了 0.5% -> 成交均价远高于他下单时的价格
- 其他交易者看到大单涌入，跟风买入 -> 价格进一步推高
- 最终多花了几万元 => 这就是市场冲击成本

TWAP 与 VWAP

TWAP -- Time-Weighted Average Price (时间加权平均价格)

把大单按时间均匀拆分 => 不管市场发生什么，每个时段执行固定的量。

Ethan 要在一天内买入 10万股，交易时间4小时(240分钟)，分成48个时段(每段5分钟):

时段	执行量	说明
09:30-09:35	2,083 股	$100,000 / 48 = 2,083$
09:35-09:40	2,083 股	同上，固定不变
...	...	每个时段都一样
14:55-15:00	2,083 股	最后一个时段也一样

优点: 实现简单，不依赖任何市场数据，执行纪律性强。

缺点: 完全不考虑市场状态。如果某个时段市场冲击特别大(比如流动性差)，TWAP 也会傻傻地执行同样的量

TWAP 与 VWAP

VWAP -- Volume-Weighted Average Price (成交量加权平均价格)

比 TWAP 进了一步: 按历史成交量的分布来拆单 => 成交量大的时段多执行, 成交量小的时段少执行。

同样买入10万股, 但根据历史数据, 上午开盘和下午收盘成交量较大:

时段	历史成交量占比	执行量	说明
09:30-09:35	5%	5,000 股	开盘量大, 多执行
10:30-10:35	1.5%	1,500 股	盘中量小, 少执行
14:30-14:35	3%	3,000 股	尾盘量回升, 适度执行
14:55-15:00	4.5%	4,500 股	收盘量大, 多执行

- 优点: 量大的时段流动性好, 冲击成本更低;执行均价更接近市场 VWAP, 是机构最常用的基准。
- 缺点: 仍然是固定规则, 不会根据当天的实时行情调整。如果今天的成交量分布和历史不一样, 效果就打折扣。

PPO 智能拆单

TWAP/VWAP 都是提前定好计划的策略

它们在执行前就确定了每个时段的量，执行过程中不会改变。

Thinking: PPO 智能拆单是怎样的？

PPO 智能拆单是实时观察市场状态，动态调整执行量 -- 冲击大时少执行，冲击小时多执行。

=> RL 拆单相对传统拆单的核心优势：从按计划执行 到 看情况执行。

DQN的局限

DQN 的局限: 动作空间必须是离散的

场景	动作空间	适合的算法
择时决策	离散: 买入/卖出/持仓 (3个)	DQN
拆单执行	连续: 执行比例 0%~100%	PPO
做市报价	离散: 5种报价策略	DQN

PPO的核心思想

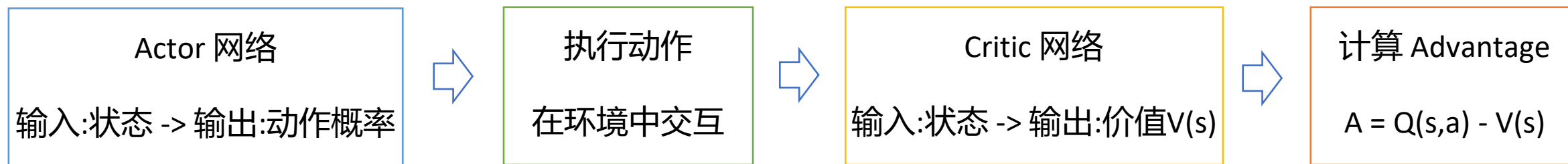
PPO 的核心思想:

PPO = Policy Gradient + 重要性采样 + Clip 限制

- Policy Gradient: 直接优化策略网络, 输出动作的概率分布
- 重要性采样 (Importance Sampling): 让旧数据可以重复利用 (从 On-policy -> Off-policy)
- Clip 限制: 防止策略更新过大导致崩溃, $\text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon)$

PPO的核心思想

Actor-Critic 架构



Actor 是演员，负责做决策；

Critic 是评论家，评价决策的好坏。

=> 两者协同训练，演员会越来越会演，评论家评价越来越准。

市场冲击模型

Thinking: 为什么大单成本高?

Almgren-Chriss 冲击模型:

临时冲击 = $\eta * \sigma * (v / ADV)^{0.5}$

永久冲击 = $\gamma * \sigma * (v / ADV)$

v = 本次执行量, ADV = 日均成交量 (约900万股)

σ = 日波动率, $\eta=0.1$, $\gamma=0.05$

平方根规律的直觉: 执行 1 万股的冲击, 不是 10 万股冲击的 1/10, 而是约 1/3。

大单的冲击是加速增长的 => 这就是拆单的理论基础。

市场冲击模型

市场冲击计算示例

total_order = 100_000 # 10万股

adv = 8_958_153 # 日均成交量

一次性执行: $\text{participation} = 100000 / 8958153 = 1.12\%$

冲击 = $0.1 * 0.02 * (0.0112)^{0.5} = 0.021\% = 2.1\text{个基点}$

分成10次执行: 每次 $\text{participation} = 0.112\%$

单次冲击 = $0.1 * 0.02 * (0.00112)^{0.5} = 0.0067\%$

总冲击 = $0.0067\% * 10 = 0.067\%$ -- 仅为一次性执行的1/3!

4-智能拆单环境.py

(MarketImpactModel 类)

CASE：智能拆单环境

环境设计：

将一个交易日分成 48 个时段（每段约5分钟），Agent 在每个时段决定执行多少。

MDP 要素	设计	维度
状态 (5维)	剩余量比例 + 剩余时间比例 + 价格偏移 + 波动率 + VWAP偏离	$[0,1]^5$
动作 (连续)	当前时段执行剩余量的比例	$[0, 1]$
奖励 (多层次)	每步: -冲击成本 - 集中度惩罚; 终端: -执行缺口	标量

CASE: 智能拆单环境

奖励函数设计 -- RL 的灵魂

```
# (1) 每步冲击成本: 成交价偏离市场价越多, 惩罚越大
impact_cost = (exec_price - market_price) / market_price
reward -= impact_cost * 50

# (2) 集中度惩罚: 单步执行超过均匀量2倍的部分额外惩罚
if exec_qty > uniform_qty * 2:
    reward -= excess * 5

# (3) 未按时完成: 临近结束但剩余量过多
if time_left < 0.2 and remaining > 0.5:
    reward -= remaining * 2

# (4) 终端: 执行缺口 (Implementation Shortfall)
reward -= abs(actual_vwap - arrival_price) / arrival_price * 30
```

Thinking: 为什么不能只有终端奖励?

如果只在结束时给一次奖励, Agent 会学到第一步就全部执行完的偷懒策略 -- 因为这样最简单, 中间没有任何惩罚。

加入每步即时惩罚 (冲击成本 + 集中度), 才能引导 Agent 学习分散执行。

4-智能拆单环境.py

(OrderExecutionEnv.step 方法, 奖励计算部分)

CASE: 智能拆单环境

运行结果:

[1] 加载数据: 510050.SH

数据量: 785 个交易日

日均成交量 (ADV): 8,958,153

订单占ADV比例: 1.12%

[2] 创建订单执行环境

时段数: 48 (每时段约5分钟)

观测空间: (5,)

动作空间: (1,)

[3] TWAP 单次执行验证

到达价格: 2.4700

执行VWAP: 2.4873

市场VWAP: 2.4865

执行缺口: 70.23 bps

VWAP滑点: 3.48 bps

分片数: 48

[4] 可视化 TWAP 执行轨迹

图表已保存: outputs/4-拆单执行轨迹.png

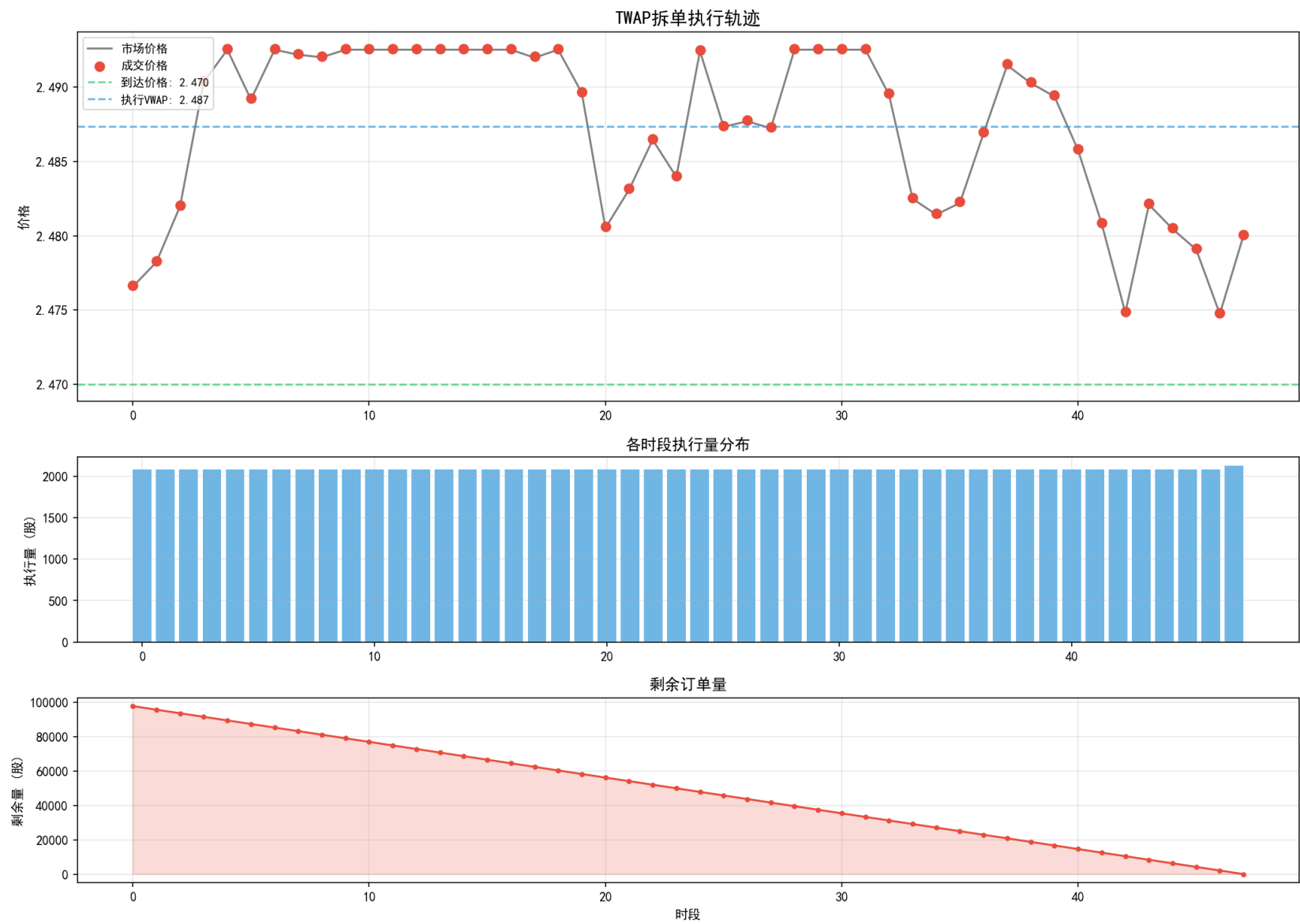
[5] 批量对比 TWAP vs VWAP (各50次)

TWAP 执行缺口: 均值=-4.83bps, 标准差=62.83bps

VWAP 执行缺口: 均值=-9.23bps, 标准差=58.24bps

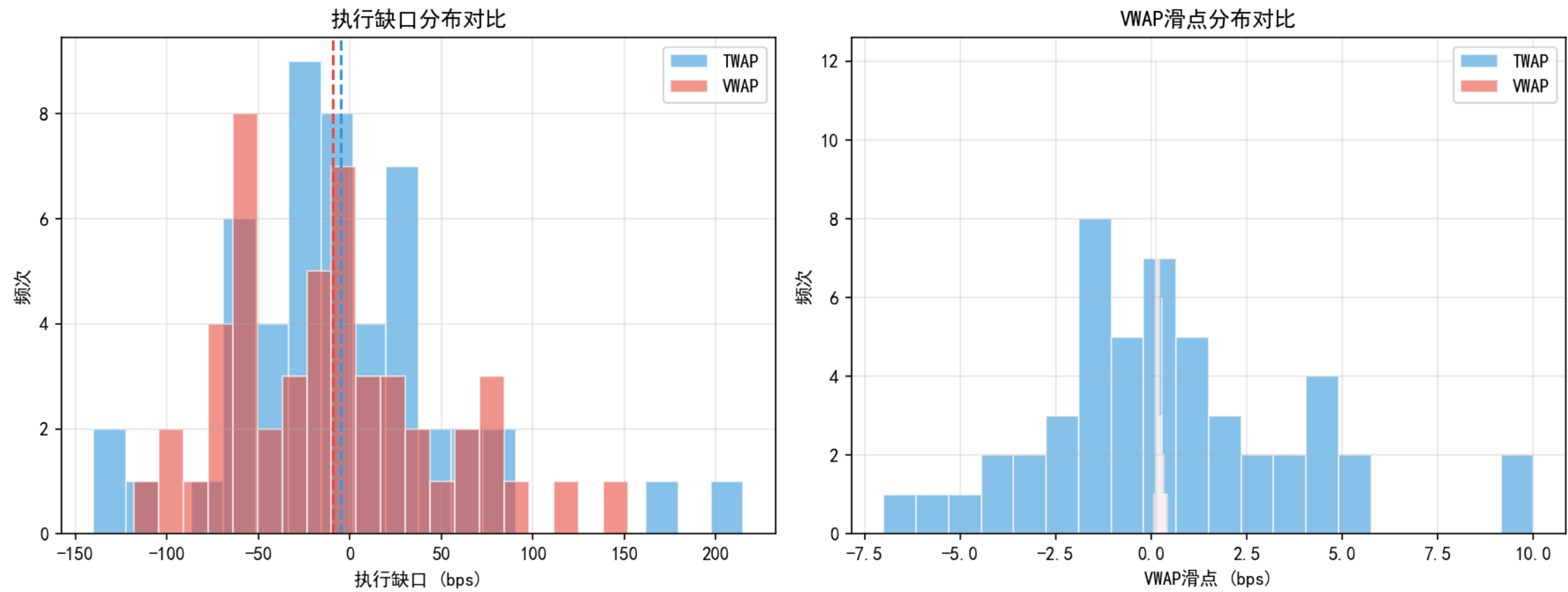
图表已保存: outputs/4-基线策略对比.png

CASE：智能拆单环境



CASE：智能拆单环境

TWAP vs VWAP 基线对比



PPO 智能拆单 vs 基线对比

PPO 训练配置

```
from stable_baselines3 import PPO

model = PPO(
    'MlpPolicy', env,
    learning_rate=3e-4,
    n_steps=2048,    # 每次收集2048步再更新
    batch_size=64,
    n_epochs=10,    # 每批数据训练10遍 (重要性采样允许重复利用)
    gamma=0.99,
    clip_range=0.2, # PPO核心: 限制策略更新幅度
    ent_coef=0.01, # 熵系数: 鼓励探索
    policy_kwargs=dict(net_arch=dict(pi=[64,64], vf=[64,64]))
)

model.learn(total_timesteps=50000)
```

5-TWAP与RL拆单对比.py

(train_ppo 函数)

PPO 智能拆单 vs 基线对比

三策略对比结果 (100次模拟)

运行结果:

策略		IS均值(bps)		IS标准差		VWAP滑点均值		分片数

TWAP		-6.44		62.00		0.09		48.0
VWAP		3.39		61.26		0.20		48.0
PPO		10.29		83.29		-5.74		10.0

PPO 相对 TWAP 执行缺口改善: 自适应分成10片执行

PPO 智能拆单 vs 基线对比

结果解读：

维度	TWAP	VWAP	PPO
分片数	48 (均匀)	48 (按量)	10 (自适应)
策略特点	固定规则	固定规则	根据市场状态动态调整
适应性	无	无	冲击大时少执行，冲击小时多执行

Thinking: PPO 学到了什么？

PPO 学会了将大单拆成约 10 片执行，而不是 TWAP 的 48 片均匀拆。它选择在市场冲击较小的时段集中执行，在冲击大的时段少执行。50000步训练已初步展现自适应能力，更多训练可进一步优化。

在实际生产环境（如 Qlib）中，PPO 会结合实时订单簿数据，效果远优于固定规则。

打卡：智能拆单



以买入10万股(约占日均成交量1.12%)为例，构建 OrderExecutionEnv 拆单环境，一天分48个时段：

- 状态空间5维(剩余量/剩余时间/价格偏移/波动率/VWAP偏离)，动作空间连续(执行比例0~1)
- 使用 Stable-Baselines3 的 PPO 算法训练50000步，与 TWAP/VWAP 基线做100次模拟对比
- PPO学会自适应拆成约10片执行(而非TWAP的48片均匀拆)，在冲击小的时段集中执行

Summary

RL 在量化交易中的应用场景

场景	RL 适配度	核心原因	谁在用
智能拆单	最高	问题结构稳定，有明确基线，奖励信号清晰	微软 Qlib、JPMorgan、Goldman Sachs
择时决策	中等偏上	市场非平稳是挑战，但 RL 风控优于传统方法	华泰金工、Two Sigma
高频做市	高 (散户用不了)	奖励清晰(价差-库存风险)，但需微秒级延迟	乾象投资、幻方量化



Thank You
Using data to solve problems

